

Методичка 7. Строки в Python

1. Зачем нужна эта тема

Строки нужны, когда программа работает с текстом.

Текст встречается почти в любой программе:

- имя пользователя;
- пароль;
- сообщение;
- команда в меню;
- слово в игре;
- ответ на вопрос;
- строка из файла;
- название товара;
- текст для обработки.

Например, если программа делает игру “Поле чудес”, ей нужно хранить слово, открывать буквы, проверять, есть ли буква в слове. Если программа проверяет пароль, ей нужно сравнивать строки. Если программа работает с сообщениями, ей нужно искать слова, заменять символы и делить текст на части.

Строки связаны с предыдущими темами:

- с переменными — строку можно хранить в переменной;
- с условиями — строки можно сравнивать;
- с циклами — строку можно перебирать по символам;
- со списками — строку можно превращать в список слов.

2. Главная идея темы

Строка — это текстовая последовательность символов.

Например:

```
word = "python"
```

Строка `"python"` состоит из символов:

```
p y t h o n
```

У каждого символа есть индекс.

Важно: индексы начинаются с 0.

```
word = "python"

print(word[0]) # p
print(word[1]) # y
print(word[2]) # t
```

Строка — текст в кавычках.

Символ — один знак внутри строки: буква, цифра, пробел, знак препинания.

Индекс — номер символа в строке.

Метод строки — готовое действие, которое можно выполнить со строкой, например `lower()`, `upper()`, `find()`.

3. Базовый синтаксис

Создание строки

```
name = "Alex"
city = 'Moscow'

print(name)
print(city)
```

Строки можно записывать в двойных или одинарных кавычках.

На что обратить внимание:

- главное — открыть и закрыть кавычки одинаково;
- без кавычек Python будет думать, что это имя переменной.

Ввод строки

```
name = input("Введите имя: ")
print("Привет,", name)
```

`input()` всегда возвращает строку.

Если пользователь вводит имя, преобразование через `int()` не нужно.

Длина строки

```
word = "python"

print(len(word))
```

Результат:

```
6
```

`len()` показывает количество символов в строке.

Получение символа по индексу

```
word = "python"

print(word[0])
print(word[3])
```

Результат:

```
p
h
```

Индексы начинаются с 0.

Перебор строки

```
word = "python"

for letter in word:
    print(letter)
```

Программа выводит каждый символ строки на новой строке.

Проверка вхождения

```
word = "python"

if "y" in word:
    print("Буква есть")
else:
    print("Буквы нет")
```

Оператор `in` проверяет, есть ли символ или подстрока внутри строки.

4. Разбор темы от простого к сложному

Шаг 1. Хранение текста

```
message = "Привет"

print(message)
```

Строка хранит текст. Её можно записать в переменную и вывести.

Шаг 2. Сравнение строк

```
password = input("Введите пароль: ")

if password == "12345":
    print("Доступ разрешён")
else:
    print("Неверный пароль")
```

Строки можно сравнивать через `==`.

Важно: `"Python"` и `"python"` — разные строки, потому что регистр букв отличается.

Шаг 3. Получение символов

```
word = "game"

print(word[0])
print(word[1])
print(word[2])
print(word[3])
```

Результат:

```
g  
a  
m  
e
```

Так можно обращаться к отдельным буквам.

Шаг 4. Перебор символов через цикл

```
word = "game"  
  
for letter in word:  
    print("Буква:", letter)
```

Цикл `for` проходит по строке от первого символа к последнему.

Шаг 5. Подсчёт букв

```
word = input("Введите слово: ")  
  
count = 0  
  
for letter in word:  
    if letter == "a":  
        count = count + 1  
  
print("Количество букв a:", count)
```

Программа перебирает все буквы и считает, сколько раз встретилась буква `"a"`.

Шаг 6. Метод `lower`

```
text = input("Введите слово: ")  
  
text = text.lower()  
  
print(text)
```

`lower()` переводит строку в нижний регистр.

Это удобно, когда нужно сравнивать текст без учёта больших и маленьких букв.

```
answer = input("Введите да или нет: ")

if answer.lower() == "да":
    print("Продолжаем")
else:
    print("Останавливаемся")
```

Шаг 7. Метод upper

```
word = "python"

print(word.upper())
```

Результат:

```
PYTHON
```

`upper()` переводит строку в верхний регистр.

Шаг 8. Метод strip

```
login = input("Введите логин: ")

login = login.strip()

print(login)
```

`strip()` убирает лишние пробелы в начале и в конце строки.

Это полезно, если пользователь случайно поставил пробел.

Шаг 9. Метод find

```
text = "я изучаю python"

index = text.find("python")

print(index)
```

`find()` ищет подстроку и возвращает индекс, где она начинается.

Если подстрока не найдена, возвращается `-1`.

Шаг 10. Метод replace

```
text = "Я люблю Java"

new_text = text.replace("Java", "Python")

print(new_text)
```

Результат:

```
Я люблю Python
```

`replace()` заменяет одну часть строки на другую.

Шаг 11. Метод `split`

```
text = "яблоко банан груша"

words = text.split()

print(words)
```

Результат:

```
['яблоко', 'банан', 'груша']
```

`split()` разбивает строку на список слов.

5. Частые ошибки учеников

Ошибка 1. Пишут строку без кавычек

Неправильно:

```
name = Alex
```

Почему это ошибка:

Python думает, что `Alex` — это переменная. Но такой переменной нет.

Правильно:

```
name = "Alex"
```

Ошибка 2. Думают, что индексы начинаются с 1

Неправильно:

```
word = "python"  
  
print(word[1])
```

Почему это ошибка:

Если нужен первый символ, индекс должен быть 0. `word[1]` — это второй символ.

Правильно:

```
word = "python"  
  
print(word[0])
```

Ошибка 3. Выходят за границы строки

Неправильно:

```
word = "cat"  
  
print(word[3])
```

Почему это ошибка:

В строке `"cat"` три символа с индексами 0, 1, 2. Индекса 3 нет.

Правильно:

```
word = "cat"  
  
print(word[2])
```

Ошибка 4. Пытаются изменить символ строки по индексу

Неправильно:

```
word = "cat"  
  
word[0] = "b"
```

Почему это ошибка:

Строки в Python нельзя изменять по символам. Можно только создать новую строку.

Правильно:

```
word = "cat"

new_word = "b" + word[1:]

print(new_word)
```

Ошибка 5. Забывают сохранить результат метода

Неправильно:

```
word = "PYTHON"

word.lower()

print(word)
```

Почему это ошибка:

Методы строк не изменяют старую строку. Они создают новую строку.

Правильно:

```
word = "PYTHON"

word = word.lower()

print(word)
```

Ошибка 6. Путают find и проверку in

Неправильно:

```
text = "hello"

if text.find("z"):
    print("Нашли")
```

Почему это ошибка:

Если символ не найден, `find()` возвращает `-1`, а это может вести себя не так, как ожидается.

Правильно:

```
text = "hello"

if "z" in text:
    print("Нашли")
else:
    print("Не нашли")
```

Или так:

```
text = "hello"

if text.find("z") != -1:
    print("Нашли")
else:
    print("Не нашли")
```

Ошибка 7. Не учитывают регистр

Неправильно:

```
answer = input("Введите да: ")

if answer == "да":
    print("Верно")
```

Почему это может быть проблемой:

Если пользователь введёт или , программа не засчитает ответ.

Правильно:

```
answer = input("Введите да: ")

if answer.lower() == "да":
    print("Верно")
```

6. Мини-примеры для закрепления

Пример 1. Длина имени

```
name = input("Введите имя: ")

print("Длина имени:", len(name))
```

Программа считает количество символов в имени. Закрепляется `len()`.

Пример 2. Первая буква

```
word = input("Введите слово: ")

print("Первая буква:", word[0])
```

Закрепляется индекс 0.

Пример 3. Перебор букв

```
word = input("Введите слово: ")

for letter in word:
    print(letter)
```

Закрепляется перебор строки циклом `for`.

Пример 4. Проверка буквы

```
word = input("Введите слово: ")

if "a" in word:
    print("В слове есть буква а")
else:
    print("В слове нет буквы а")
```

Закрепляется оператор `in`.

Пример 5. Подсчёт символов

```
word = input("Введите слово: ")

count = 0

for letter in word:
    if letter == "o":
        count = count + 1

print("Букв o:", count)
```

Закрепляется цикл, условие и счётчик.

Пример 6. Очистка пробелов

```
login = input("Введите логин: ")

login = login.strip()

print("Логин:", login)
```

Закрепляется `strip()`.

Пример 7. Замена слова

```
text = "Я изучаю Java"

text = text.replace("Java", "Python")

print(text)
```

Закрепляется `replace()`.

Пример 8. Разделение строки

```
text = input("Введите несколько слов: ")

words = text.split()

print(words)
```

Закрепляется `split()`.

7. Практические задания

Лёгкий уровень

1. Пользователь вводит слово. Вывести его длину.
2. Пользователь вводит слово. Вывести первую букву.
3. Пользователь вводит слово. Вывести каждую букву на новой строке.
4. Пользователь вводит слово. Проверить, есть ли в нём буква `"a"`.
5. Пользователь вводит строку. Перевести её в нижний регистр.

Средний уровень

1. Пользователь вводит слово. Посчитать количество букв `"o"`.
2. Пользователь вводит строку. Убрать лишние пробелы в начале и конце.
3. Пользователь вводит строку. Заменить все пробелы на дефисы.
4. Пользователь вводит предложение. Разбить его на список слов.
5. Пользователь вводит пароль. Проверить, что его длина не меньше 8 символов.

Повышенный уровень

1. Пользователь вводит строку. Посчитать количество гласных букв.
2. Пользователь вводит слово. Проверить, одинаковая ли первая и последняя буква.
3. Пользователь вводит предложение. Посчитать количество слов.

8. Контрольные вопросы

1. Что такое строка?
2. Что такое символ?
3. С какого индекса начинается строка?
4. Что делает `len()` ?
5. Можно ли изменить символ строки по индексу?
6. Что делает метод `lower()` ?
7. Что делает метод `upper()` ?
8. Что делает метод `strip()` ?
9. Что делает метод `find()` ?
10. Что возвращает `find()` , если ничего не найдено?
11. Что делает `replace()` ?
12. Что делает `split()` ?

9. Краткий конспект в конце

Главная идея темы: строка хранит текст и позволяет работать с каждым символом отдельно.

Основные конструкции:

```
text = "python"
```

```
print(text[0])
```

```
print(len(text))
```

```
for letter in text:  
    print(letter)
```

```
text = text.lower()
```

```
words = text.split()
```

Важно запомнить:

- строка записывается в кавычках;
- индексы начинаются с 0;
- строку можно перебирать циклом;
- строки нельзя изменять по символам;
- методы строк возвращают новую строку;
- `in` удобно использовать для проверки наличия символа или слова.

Частые ошибки:

- забыть кавычки;
- обратиться к несуществующему индексу;
- пытаться изменить символ строки;
- забыть сохранить результат метода;
- не учитывать регистр букв.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать строки;
- вводить строки с клавиатуры;
- получать длину строки;
- обращаться к символам по индексу;
- перебирать строку циклом;
- проверять наличие символа;
- использовать методы `lower`, `upper`, `strip`, `find`, `replace`, `split`;
- решать простые задачи на обработку текста.